

Realizar la implementación distribuida con paso de mensajes (MPI) de acuerdo a los fundamentos vistos en clase y medir el SpeedUp para 2, 4 y 8 unidades de ejecución

Es obvio que la ejecución con MPI deberá tardar menos que la solución seria

1. Ordenamiento de vector (por facilidad el vector tiene aleatorios)

OUTPUT:

```
Enter number of elements you want to sort: 6
```

```
5 3 4 2 1 6
```

```
1 2 3 5 6
```

2. Remueve los elementos duplicados de un vector desordenado ((por facilidad el vector tiene aleatorios)

OUTPUT:

```
10
```

```
1 2 2 3 4 5 6 7 7 8
```

```
Array Before Removing Duplicates: 1 2 2 3 4 5 6 7 7 8
```

```
Array After Removing Duplicates: 1 2 3 4 5 6 7 8
```

3. Dada una matriz chequea si una matriz es sparse ((por facilidad la matriz tiene aleatorios)

The entered matrix is :

2	0	4
0	0	8
0	6	0

The entered matrix is a sparse matrix

4. Dada una matriz chequea si una matriz es simétrica

Enter the 9 elements of the matrix

1 7 3 7 4 -5 3 -5 6

The original matrix is :

1	7	3
7	4	-5
3	-5	6

The Transpose matrix is :

1	7	3
7	4	-5
3	-5	6

Matrix is Symmetric

5. Dada 2 matrices realiza su suma

```

Enter the 9 elements of the first matrix
1 4 3 6 7 4 0 8 4

Enter the 9 elements of the second matrix
2 4 5 9 6 7 0 3 4

The first matrix is :
1      4      3
6      7      4
0      8      4

The second matrix is :
2      4      5
9      6      7
0      3      4

The sum of the two entered matrices is :
3      8      8
15     13     11
0      11     8

```

6. Realiza la suma de los primeros n números naturales

```
Input Value of terms : 7
```

```
The first 7 natural number is :
```

```
1 2 3 4 5 6 7
```

```
The Sum of Natural Number upto 7 terms : 28
```